

DAY 4 — PLC ADVANCED CONCEPTS

Shift Registers • Sequencing • FB/FC • Mode Logic • PID



PLC + HMI training context

Track

Sequence

Reuse

Select Mode

Control

Visual trainer edition with diagrams, industrial examples, student exercises and troubleshooting.

Training Roadmap

What Day 4 teaches students to build and explain



- Understand the PLC scan/cycle and why it affects events, timers and PID.
- Track moving products with clean shift/index logic.
- Build sequence flow using state, action and transition thinking.
- Use FBs, FCs and instance data to keep programs scalable.
- Use Manual/Auto/Stop/Fault mode logic to prevent command conflict.
- Explain PID as closed-loop control, including P, I, D and cycle-time effect.

OVERVIEW

Where Day 4 Fits in a PLC Project

From individual instructions to machine architecture

- Day 1–3 instructions are building blocks.
- Day 4 is about coordinating those blocks into a clear, safe and reusable machine structure.
- Architecture decides who owns commands and how data flows between modules.



Real machine control has layers

Inputs

Conditioning

Mode

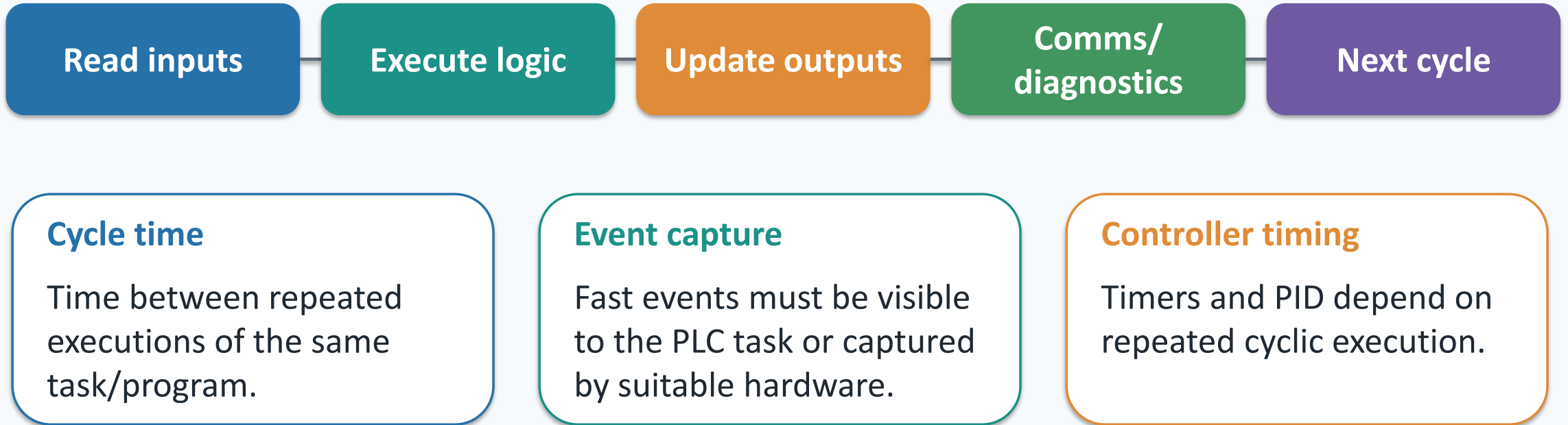
Sequence

Equipment
FBs

Outputs

PLC Scan Cycle

Foundation for shift registers, sequences and PID



- Important: scan/cycle time is not production shift timing such as morning/afternoon shift.

Scan Time vs Machine Event

Why a sensor should not be treated as a perfect instant

Example PLC task

Cycle time = 10 ms
Approx. 100 scans per second

Example sensor event

Product blocks sensor for 150 ms
PLC may see it many times

Teaching point

A raw TRUE input can last for many scans. Use an edge/one-shot when the machine event must count or shift once.



Sensor pulse must be captured

Cycle Time Effects

Same logic can behave differently when timing changes

Shift registers

Shift once per index event, not once per scan.

Sequencing

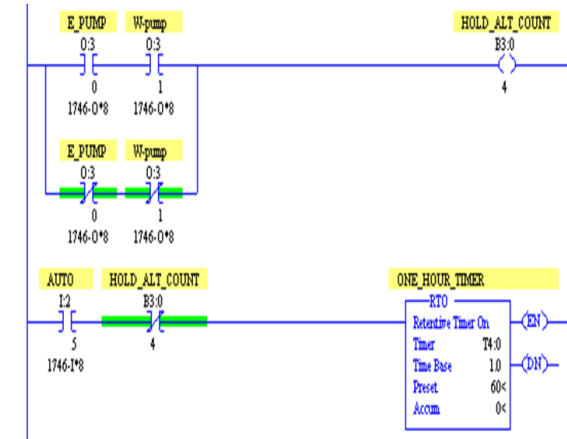
State transitions are evaluated every cycle.

Timers

Elapsed-time blocks update through cyclic calls.

PID

Controller math depends on sampling/execution interval.



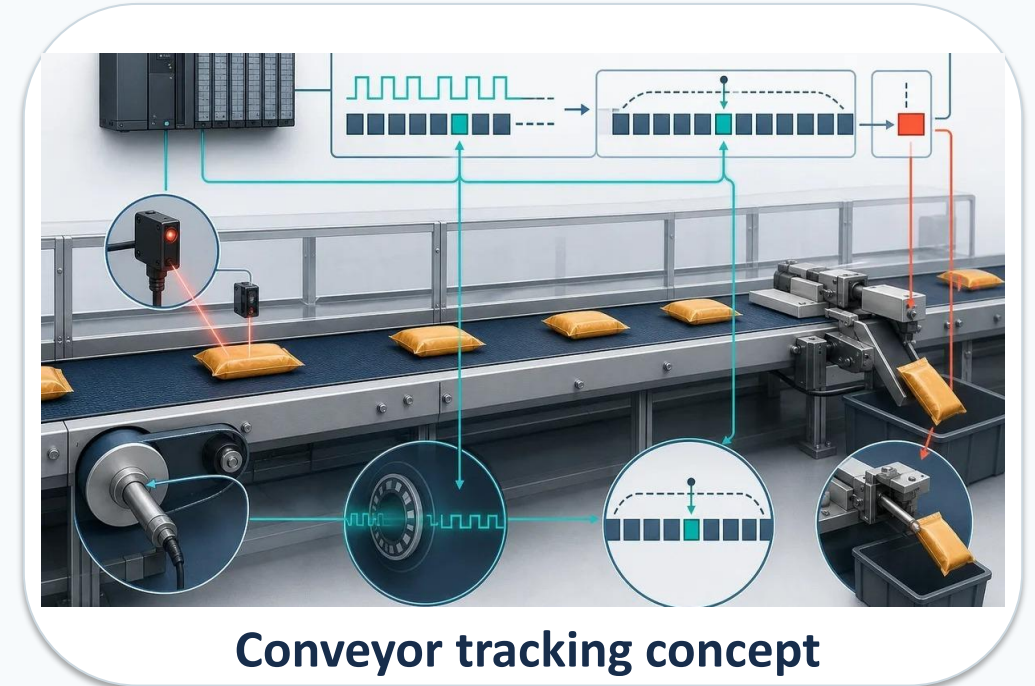
Online monitoring and task timing matter

SHIFT

Shift Register — Core Idea

Track product information as the product moves

- A shift register stores product/status information in logical positions.
- Each valid index pulse moves the stored information one position.
- The memory position must match the physical product position.



Entry

Pos 1

Pos 2

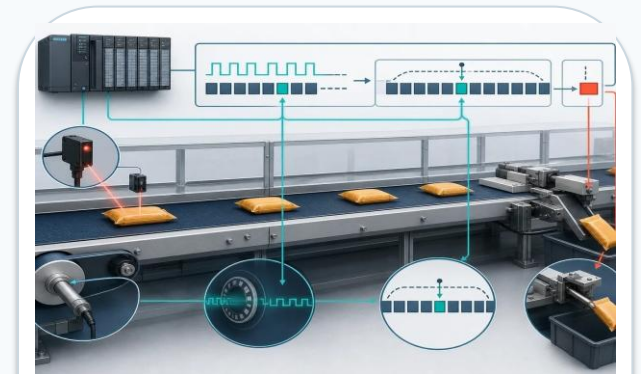
Pos 3

Reject

Bit Movement Example

One defective product moving toward the reject station

Event	B0 Entry	B1	B2	B3 Reject
Initial	0	0	0	0
Defect detected	1	0	0	0
Index 1	0	1	0	0
Index 2	0	0	1	0
Index 3	0	0	0	1 → pusher



Bit follows product

Key point

The bit is not the product; it is the PLC's remembered information about the product.

SHIFT

Correct Shift Triggering

A real machine event must become one clean PLC event

Wrong

Shift while sensor = TRUE.

If sensor stays TRUE for 5 scans, data may shift 5 times.

Correct

Create a one-shot/rising-edge.
One physical index → one logical shift.



Photoeye/index sensor

Sensor
signal

Rising
event

Shift once

Clear event

SHIFT

Shift Register Flowchart

Graphical logic without Ladder or ST

Wait for index

Detect edge

Shift existing
data

Insert new data

Use station data

Input event

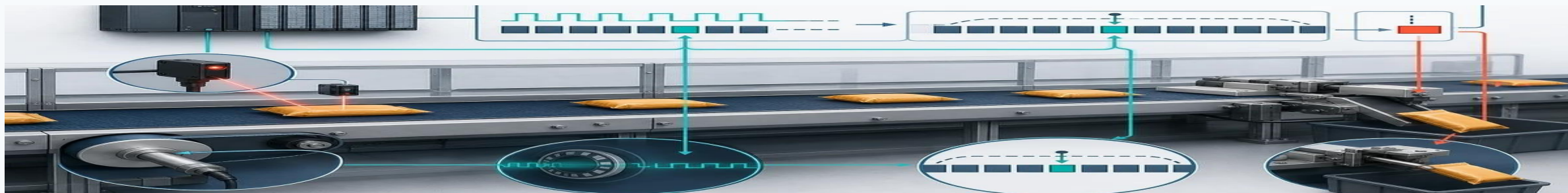
Index pulse or
encoder pitch
indicates product
movement.

Memory action

Stored bits or
product records
move forward.

Output decision

Station bit triggers
pusher, diverter or
packaging.



Common Shift Register Faults

Troubleshooting with scan-time thinking

Multiple shifts

Trigger is level-based instead of edge-based.

One station offset

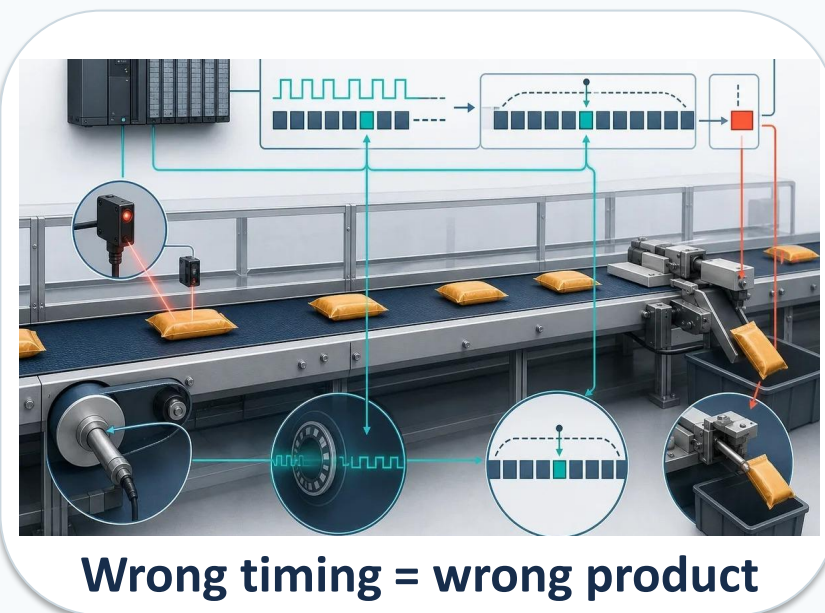
Physical station and memory index do not match.

Wrong direction

Data shifts opposite to product flow.

Lost tracking

Conveyor slip or missing index pulse breaks alignment.



SEQUENCE

Program Sequencing

State, action and transition



- A state tells what the machine is currently doing.
- Actions are outputs or commands allowed in that state.
- Transitions are conditions that allow the machine to move to the next state.
- Any important fault can force the sequence into a defined Fault/Recovery state.

Sequence Anatomy

Students should identify these three parts in every step

STATE

Example: MIXING

This is the PLC memory of the active step.

ACTION

Example: Mixer_RunCmd = TRUE

What the machine is allowed to do now.

TRANSITION

Example: MixTimerDone AND NoFault

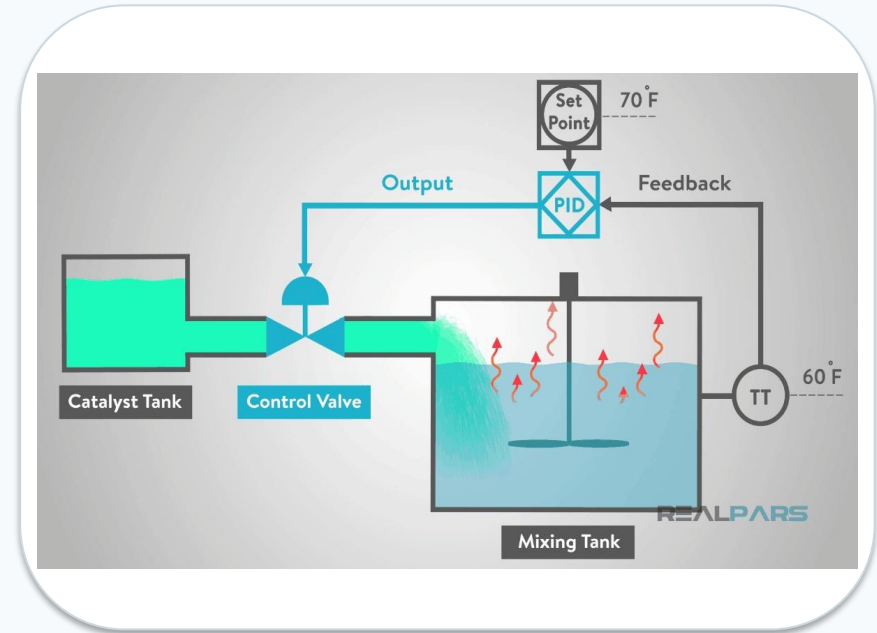
What proves the step can finish.

SEQUENCE

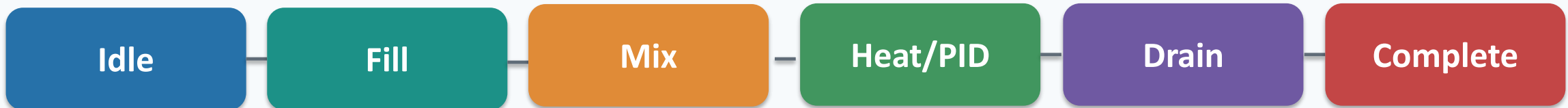
Batch Mixing Sequence

Worked example for the Day 4 project

- Fill until level feedback confirms the tank is full.
- Mix for the required process time.
- Enable PID control during the heating/holding state.
- Drain until low-level feedback confirms completion.
- Timeout any step that does not receive expected feedback.



Tank process example



SEQUENCE

Sequence Fault Path

A professional sequence includes abnormal paths



Fault path

At any state: critical fault → remove commands → latch fault → show reason → require recovery/reset.

- Without a fault path, a sequence can appear 'stuck' with no teaching or diagnostic value.
- Students should always ask: what if the expected sensor never changes?

Process Timer vs Timeout Timer

They look similar but solve different problems

Process timer

Intentional duration required by the process.

Examples:

- Mix for 10 s
- Heat hold for 30 s
- Settle for 2 s

Timeout timer

Maximum allowed time before declaring a fault.

Examples:

- Valve did not open
- Level did not reach high
- Cylinder did not return home

Teaching point: timers are not only delays; they are also diagnostic tools.

SEQUENCE

Sequence Troubleshooting Map

A repeatable method for students



- Do not randomly force outputs first.
- Look at the active state and ask what is expected in that state.
- Verify each transition condition individually.
- Check if an edge condition, timer done bit or feedback signal is missing.
- Confirm the sequence is not being forced to Fault or Stop by mode logic.

Function Block (FB)

Reusable equipment behavior with memory

- An FB packages behavior and internal state.
- Each instance has its own memory.
- Best for motors, valves, cylinders, conveyors, alarms and PID wrappers.



Reusable modules inside a PLC project

Instance Data

Same FB code, separate memory per machine object

Motor_FB type

Common behavior:
Start/stop
Interlock
Fault latch
Status generation

Motor_01 instance

Own commands
Own feedback
Own timers
Own fault status

Motor_02 instance

Same code
Different memory
Different I/O

Instance data prevents repeated equipment from accidentally sharing internal state.

Function (FC)

Reusable calculation without equipment memory focus

Good FC examples

- Scaling calculation
- Unit conversion
- Limit checking
- Reusable formula
- Small stateless logic

Remember the difference

FB = behavior + instance memory.

FC = calculation/reusable logic without a separate equipment instance concept.

Input values

Function

Result

FB vs FC vs Program

Choosing the right software object

Object	Best used for	State / Memory	Example
Program	Top-level cyclic coordination	Application-level	Main machine call structure
FB	Reusable equipment/control behavior	Per instance	Motor, valve, PID wrapper
FC	Reusable calculation or conversion	No equipment instance	Scaling, math, validation

Program Organization

Keep command ownership clear

- I/O mapping isolates hardware addresses.
- Mode manager decides who is allowed to command equipment.
- Sequence creates automatic requests.
- Equipment FBs enforce local logic and produce status.



Organized PLC/HMI system

I/O Mapping

Mode Manager

Sequence

Equipment FBs

Output Mapping

Recommended Day 4 Project Structure

Useful for batch mixing and future capstone projects

PRG_Main

Top-level calls and execution order.

PRG_IO

Maps hardware to readable tags.

FB_ModeManager

Manual/Auto/Stop/Fault ownership.

FB_Sequence

Automatic state machine.

FB_Valve / Motor

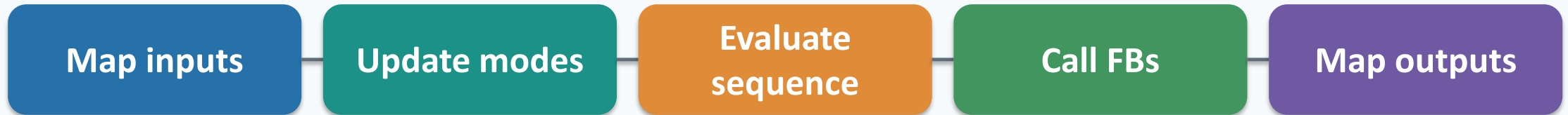
Reusable equipment modules.

FC_Uilities

Scaling, conversion, calculations.

Execution Order Matters

Visual logic still has dependency order



- If Module B uses results from Module A, the call order must be intentional.
- Graphical CFC/FBD layout should not hide data dependencies.
- Avoid writing one physical output from multiple unrelated places.
- One clear command path is easier to teach, test and troubleshoot.

MODE

Mode Logic

Manual, Auto, Stop and Fault should not fight each other

- Mode logic defines who owns equipment commands.
- Manual mode is for controlled testing; Auto mode is for sequence control.
- Fault and Stop normally have higher priority than Manual or Auto.



HMI mode selection needs PLC ownership logic

Request

Validate

Select owner

Generate
command

Equipment FB

Operating Modes

Suggested behavior to teach students

STOP

Remove normal motion commands. Allow reset/diagnostics as designed.

MANUAL

Operator jogs individual equipment through permissives/interlocks.

AUTO

Sequence manager owns normal process commands.

FAULT

Inhibit commands; show reason; require recovery/reset.

Mode Selection Flowchart

Priority logic example



- Priority example: Fault > Stop > Manual > Auto.
- Requested mode and active mode are different.
- Auto becomes active only when permissives are valid.
- Leaving Auto should have defined behavior: hold, abort or reset the sequence.

MODE

Manual Command Path

Manual does not mean bypass protection



- Manual jog command is a request, not a direct output.
- Interlocks and protection logic still apply.
- The equipment FB should make the final local decision.



Operator request must still be checked

Auto Command Path

The sequence owns commands only in Auto



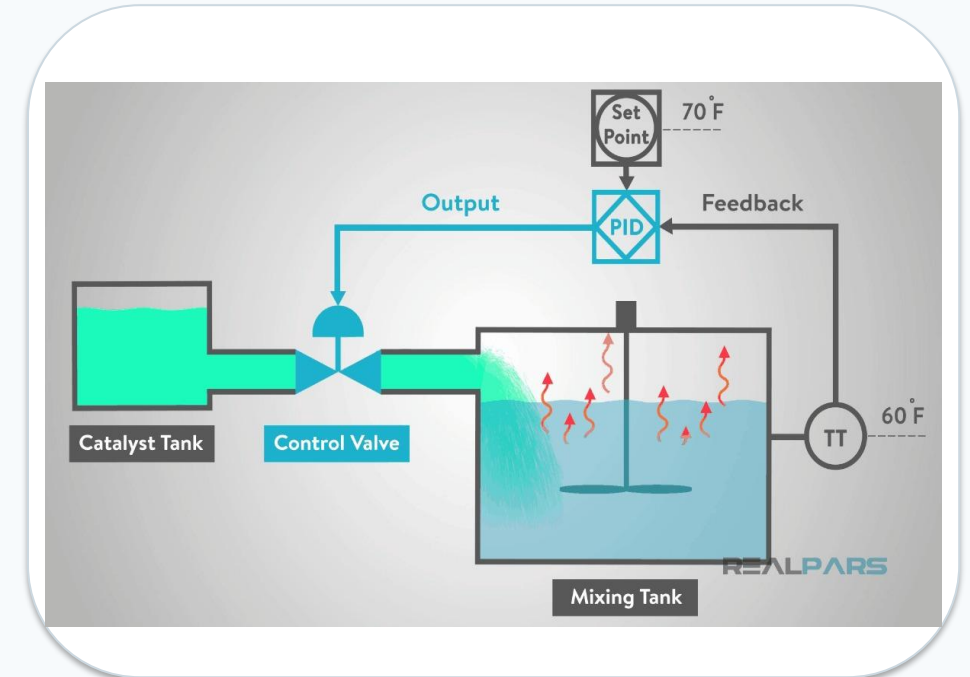
- The state machine creates requests based on the current step.
- Mode logic allows those requests only when Auto is active.
- Equipment FB evaluates feedback, local interlocks and faults.
- This chain prevents scattered commands and surprise motion.

PID

PID Basics

Closed-loop control for continuous process variables

- PID continuously adjusts an output to reduce error.
- It is used for temperature, pressure, flow, level and speed control.
- Feedback makes it closed-loop control.



Closed-loop tank temperature example



PID Vocabulary

Use correct words before tuning

SP

Setpoint — desired target
Example: 60 °C

PV

Process Variable — measured value
Example: 52 °C

Error

SP – PV
Difference
controller reacts to

MV

Manipulated Variable/output
Heater %, valve %, speed ref



PID vocabulary connects
measurement and actuator

P, I and D — Intuition

Teach the role of each action separately

P: Present error

Responds immediately to current error.

Too low: sluggish.
Too high:
overshoot/oscillation.

I: Past error

Accumulates persistent error over time.

Removes offset but can cause windup/overshoot.

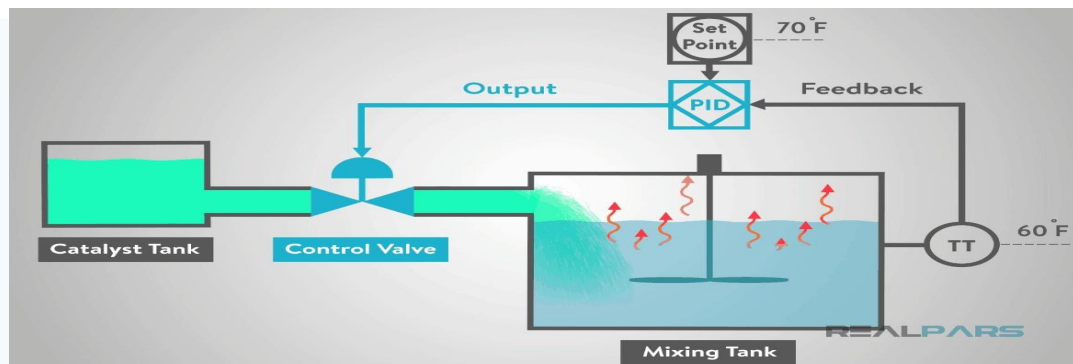
D: Rate of change

Responds to how quickly the error/PV is changing.

Can damp movement but is noise-sensitive.

PID Response Signs

Observed behavior	Possible reason	Teaching direction
Slow approach to target	Action too weak or process is slow	Increase responsiveness carefully
Overshoot	Controller too aggressive	Reduce gain/integral effect
Continuous oscillation	Poor tuning or process delay	Retune and check process dynamics
Steady offset	Insufficient integral action	Add/adjust I carefully
Noisy output	Noisy PV or derivative sensitivity	Filter PV or reduce D



PID tuning is process-dependent

PID

PID Cycle Time

Why scan/cycle time matters to controller behavior

PID

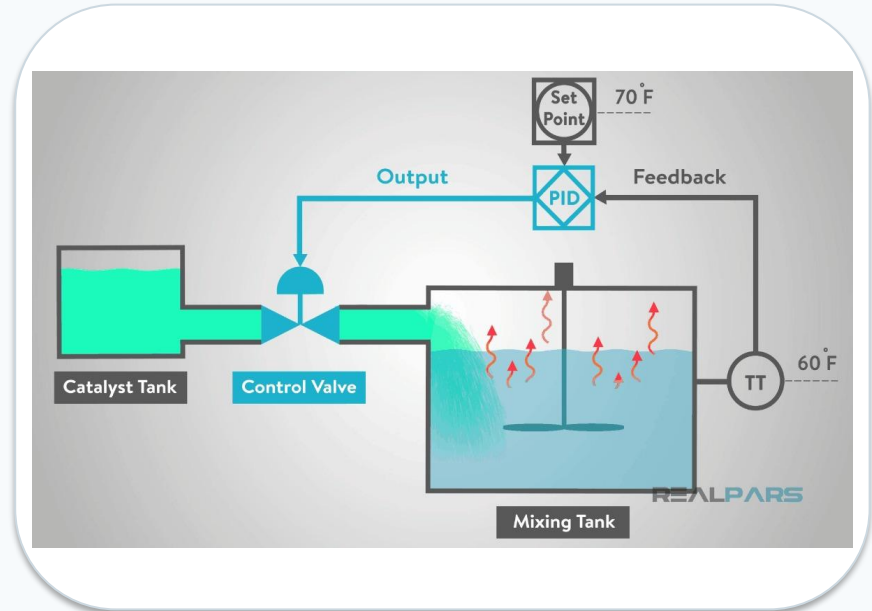
Controller uses elapsed time between calls. Very short/variable cycles can affect behavior.

Fixed-cycle PID

Uses an explicit configured cycle interval; useful when the task has a known fixed cycle.

point

A PID controller is mathematical, but in a PLC it is still executed by software at a task/cycle interval.



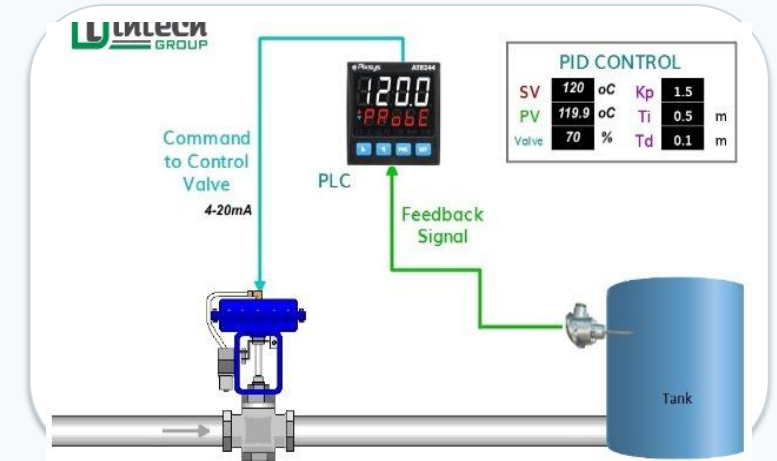
Controller executes cyclically

Manual / Auto Around PID

Mode logic must handle controller ownership



- Manual: operator/commissioning value commands the actuator.
- Auto: PID output commands the actuator.
- Transfer between Manual and Auto should avoid sudden output jumps.
- Limits protect the actuator/process range.



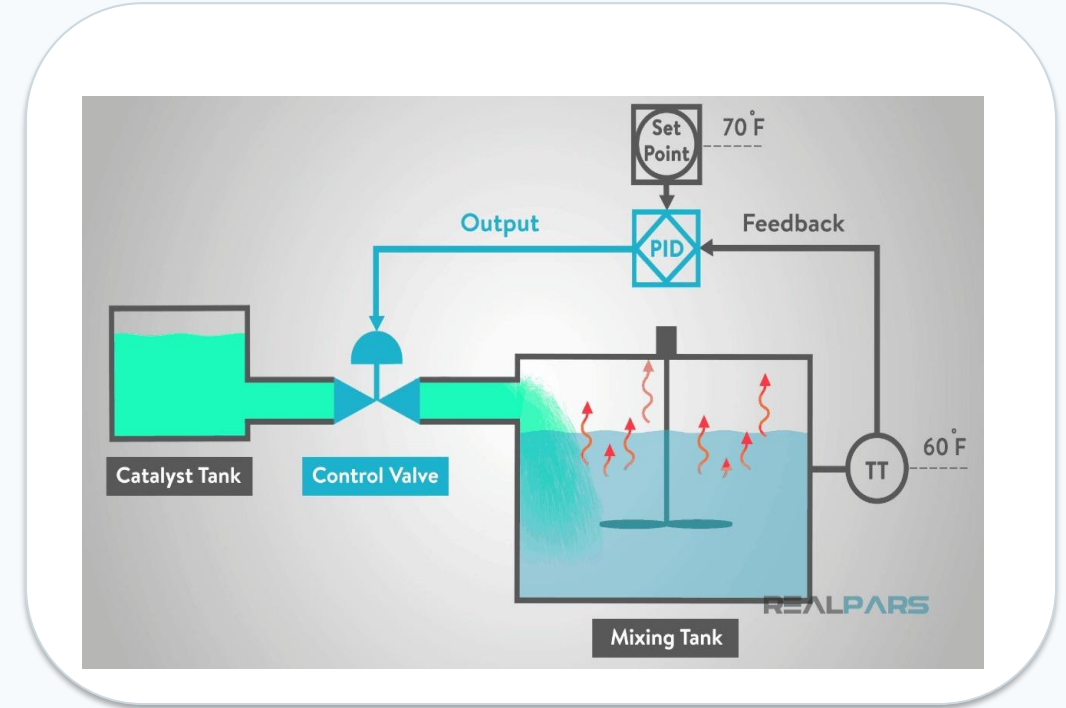
Valve/heater command path

INTEGRATION

Integrated Batch Mixing Example

Combining sequencing, mode logic, FBs and PID

- Auto sequence owns valves, mixer and heating enable.
- Manual mode allows permitted jogs through FBs.
- PID regulates temperature during the correct step.
- Fault logic removes commands and requires recovery.



Batch mixing + temperature control

Select mode

Fill

Mix

Heat/PID

Drain

Complete

Integrated Program Modules

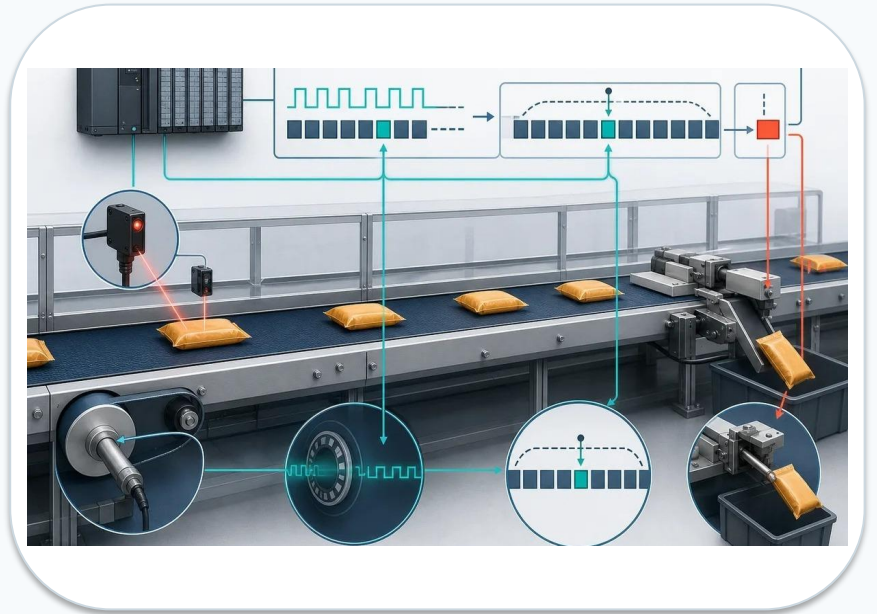
Module	Responsibility	Example data
I/O mapping	Hardware to readable tags	LevelHigh, TempPV, HeaterCmd
Mode manager	Command owner selection	ManualActive, AutoActive
Sequence	Step progression	Fill → Mix → Drain
Valve/Motor FBs	Local equipment behavior	Cmd, feedback, fault
PID wrapper	Continuous temperature control	SP, PV, Output

EXERCISE

Student Exercise — Shift Register

Use this as classroom activity

- Draw a 5-position tracking register.
- Mark a defective product at entry position.
- Show the bit pattern after four index pulses.
- Identify which bit should trigger the reject pusher.
- Explain why a rising-edge index is used.



Activity context

EXERCISE

Student Exercise — Sequence Flowchart

No Ladder or ST needed

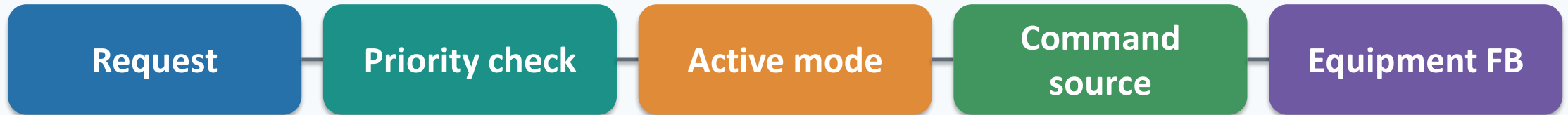


- Write one action for each state.
- Write one transition for each state.
- Add one timeout fault.
- Add a Fault state and Reset path.
- Decide where mode logic should block commands.

EXERCISE

Student Exercise — Mode Logic

Command arbitration challenge

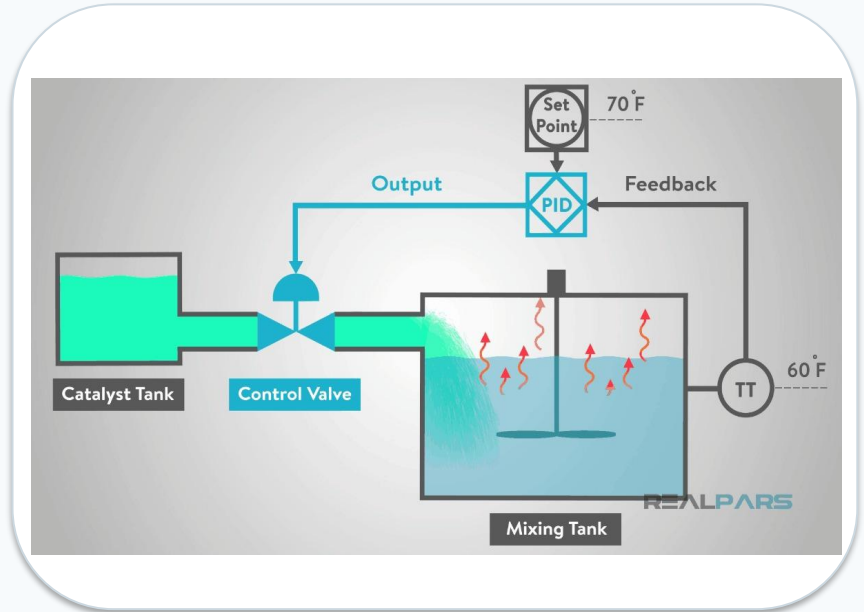


- Define Stop, Manual, Auto and Fault mode behavior.
- Choose a priority order.
- Show how ManualJog and AutoRunRequest become one MotorCommand.
- List interlocks that remain active in Manual mode.
- Explain what should happen when changing Auto → Manual.

EXERCISE

Student Exercise — PID Thinking

- $SP = 60\text{ }^{\circ}\text{C}$, $PV = 50\text{ }^{\circ}\text{C}$. What is the error?
- What should proportional action generally do to heater demand?
- Why does integral action help when PV remains low for a long time?
- Why can derivative action become troublesome with noisy signals?
- Why does controller execution/cycle time matter?



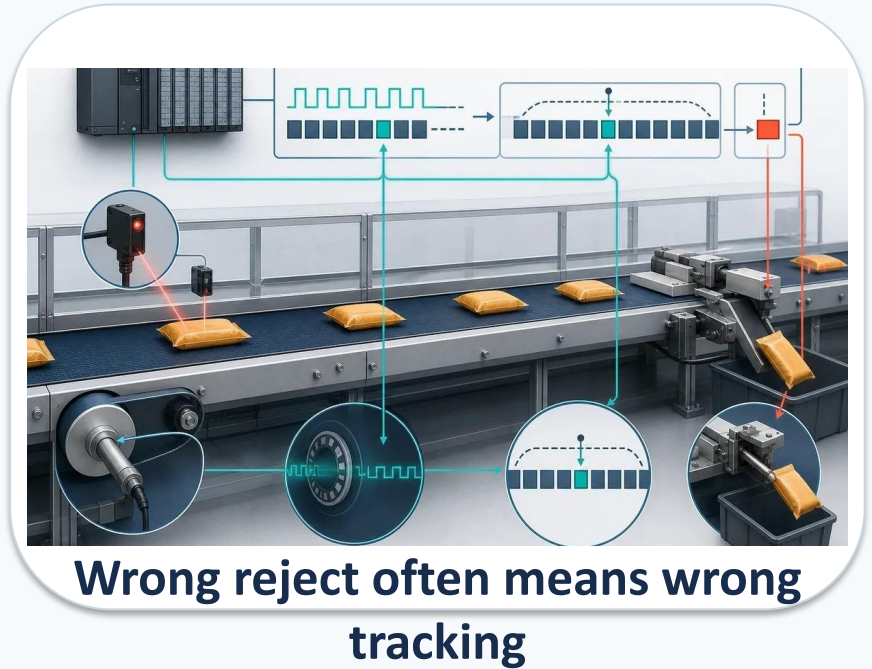
Temperature-control discussion

TROUBLESHOOTING

Troubleshooting — Tracking Is Wrong

Use this path before changing code randomly

- Multiple wrong shifts usually point to missing edge logic.
- Consistent one-station early/late points to physical-to-memory offset.
- Random misses may point to pulse duration or sensor reliability.



Wrong reject

Check index pulse

Check shift direction

Check station offset

Check sensor mapping

Troubleshooting — Auto Will Not Start

Mode logic and permissives first



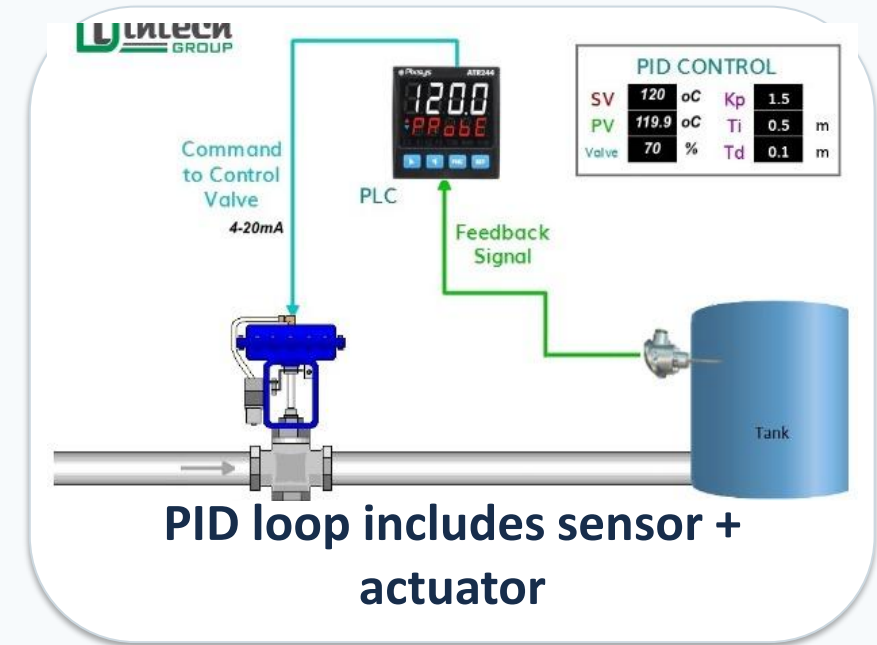
- Check requested mode vs active mode.
- Inspect fault, stop and safety/permissive conditions.
- Verify the sequence is in a valid start or recovery state.
- Check equipment FB readiness and feedback.
- Do not bypass mode logic just to make the output energize.

TROUBLESHOOTING

Troubleshooting — PID Output Looks Rough

Check the whole loop, not only tuning

- Confirm PV is scaled correctly.
- Check measurement noise and filtering.
- Check how often the controller is called.
- Check output saturation and actuator limits.
- Only then adjust tuning.



PV scaling

PV noise

Cycle time

Tuning

Output limits

REVIEW

- Why can a level-triggered shift operation move data multiple positions?
- What is the difference between a state and a transition?
- Why do multiple FB instances need separate instance data?
- When would you choose an FC instead of an FB?
- Why should Manual and Auto not write directly to the same physical output?
- What does P, I and D each respond to?
- Why can PID cycle time affect controller behavior?
- What is the difference between scan/cycle time and production shift timing?

Day 4 Recap



- Shift registers synchronize stored product information with movement.
- Sequences make machine steps visible and diagnosable.
- FBs create reusable stateful behavior; FCs centralize calculations.
- Program organization establishes command ownership and execution flow.
- Mode logic prevents Manual/Auto/Fault conflicts.
- PID introduces continuous closed-loop control where cycle time matters.